SYMBIOSIS
ARTIFICIAL INTELLIGENCE INSTITUTE
।वसुधैव कुटुम्बकम्॥

Python Project-Based Learning "Mini Project"

- Build a Python Application

SMART RECIPE RECOMMEDER

SUBJECT- PYTHON PROGRAMMING

DEPT- BBA AI Section B

SEMESTER- 1

Submitted by –

Vedic Singhal 25030422130

Sai Shan Logani 25030422103

Shreyas R. Nair 25030422115

Submitted to –

Dr. Dawa Lepcha

Assistant Professor

SAII, SIU

Submitted on: 5th November 2025

CERTIFICATE

This is to certify that the project work entitled "python project-based learning "mini project"- build a python application" on

SMART RECIPE RECOMMENDER that is submitted by

VEDIC SINGHAL (PRN:25030422130)

SAI SHAN LOGANI (PRN:25030422103)

SHREYAS R. NAIR (PRN:25030422115

in partial fulfillment for the award of degree of Bachelor of Science in ARITIFICIAL INTELLIGENCE is a record of bonafide work carried out by them. The results embodied in this report have been verified and found satisfactory.

ACKNOWLEDGMENT

We would like to express our heartfelt gratitude to Mr. Dawa Lepcha, our respected teacher, for his valuable guidance, support, and encouragement throughout the completion of our Python mini project — *SMART RECIPE RECOMMENDER*. His constant motivation and insightful feedback have greatly enhanced our understanding of programming and helped us successfully complete this project.

We would also like to thank our classmates and friends for their cooperation, and our families for their continuous encouragement and support during the course of this work.

We would also like to thank our Director Ms .Shruti Patil for her valuable insights.

This project was developed as part of the Python Programming subject under the BBA in Artificial Intelligence program at Symbiosis Artificial Institute, as a requirement for Semester 1 Mini Project.

VEDIC SINGHAL (PRN:25030422130

SAI SHAN LOGANI(PRN:25030422103)

SHREYAS R. NAIR(PRN:25030422115)

DECLARATION

We, the undersigned students of **BBA in Artificial Intelligence** at **Symbiosis Artificial Institute**, hereby declare that the mini project titled **"SMART RECIPE RECOMMENDER"**
has been carried out by us as part of **the Python Programming** subject.

We affirm that this project is the result of our own independent work and has not been submitted previously, in part or in full, for any other course or institution. All sources of information and references used have been duly acknowledged.

We have worked collaboratively and contributed equally to the successful completion of this project.

VEDIC SINGHAL (PRN:25030422130)

SAI SHAN LOGANI (PRN:25030422103)

SHREYAS R. NAIR (PRN:25030422115)

Date: 5TH Nov 2025

Guide:

Dr. Dawa Lepcha

Assistant Professor

SAII, SIU

ABSTRACT

The Smart Recipe Recommender is a Python-based application that suggests recipes based on the ingredients available to the user. It features an extensive categorized recipe database across multiple cuisines and employs matching logic to identify dishes that can be prepared with given inputs. The project demonstrates the potential of automation in everyday cooking through efficient data handling and user interaction in a command-line environment.

INTRODUCTION

Background:

The field of computer programming provides a foundation for developing interactive applications that combine logical reasoning with user engagement. In today's digital age, artificial intelligence and automation are transforming daily life activities. The Smart Recipe Recommender aims to simplify cooking by suggesting possible recipes from a predefined database based on the user's available ingredients. The system promotes creativity, minimizes food wastage, and helps individuals plan meals efficiently.

Motivation:

The motivation behind this project stems from the desire to apply core programming principles—such as modular design, conditional logic, iterative control, and user input handling—in a practical and engaging context. Smart Recipe Recommender helps a person to easily get a recipe with the available products with them.

Objectives:

The primary objectives of this project is to:

1. Design and implement a fully functional, application which can be used easily and day to day.
2. Apply structured programming techniques to ensure readability, maintainability, and efficiency.
3. Incorporate error handling, win-condition detection, and draw identification to enhance reliability.
4. Provide a user-friendly interface through clear prompts and interactive user interface.

Scope of the Project:

The scope of the project is limited to a text-based version of an application which provides or suggests various recipes with the ingredients provided.

This implementation serves as a foundation for future enhancements, such as the integration of artificial intelligence or graphical user interfaces (GUI), illustrating the scalability and flexibility of Python-based application development.

Problem Statement:

Many individuals struggle with deciding what to cook based on the ingredients they have at home. This project addresses this problem by providing intelligent recipe recommendations that adapt to ingredient availability.

Several studies and implementations have explored the development of strategy-based games like *Connect Four* as a means to apply computational logic and artificial intelligence.

Earlier works primarily focused on graphical or web-based versions using libraries such as Pygame, Tkinter, or JavaScript frameworks to enhance user interaction. Research on game algorithms, particularly the Minimax and Alpha-Beta pruning techniques, has also been widely applied to enable computer-controlled opponents in *Connect Four* and similar games.

Educational projects and tutorials often emphasize console-based implementations as foundational exercises for learning programming logic, data structures, and control flow.

These implementations highlight the importance of modular design, user input validation, and efficient condition checking.

Compared to existing studies, this project focuses on creating a simple, text-based version that prioritizes logic accuracy, code readability, and ease of execution, serving as a foundation for future enhancements such as graphical interfaces or AI integration.

METHODOLOGY and IMPLIMENTATION

The methodology involves designing a Python program capable of reading user input, processing it against a structured recipe dataset, and returning relevant suggestions.
The project applies fundamental concepts of Python such as dictionaries, loops, list comprehensions, and conditional statements.
Steps Involved:

1. **User Input:** The user enters a list of available ingredients in the console.

2. **Recipe Database:** The program stores categorized cuisines and their recipes in a nested dictionary format.

3. **Matching Algorithm:** It checks for overlapping ingredients between the user's input and stored recipes.

4. **Output Display:** The program displays matched recipes, missing ingredients, and their cuisine categories.

5. **Result Generation:** Results are printed in a clean, organized command-line format.
This process ensures efficient ingredient matching and provides flexibility even with limited input.
The modular code design allows easy expansion of the recipe database and adaptation for future GUI or AI-based interfaces.

Software and Tools:

- **Programming Language:**
  The project is developed using **Python 3**, chosen for its readability, simplicity, and extensive support for procedural programming. Python's built-in data structures and control flow constructs make it ideal for implementing game logic efficiently.

- **Development Environment:**
  Code development and testing were performed in editors such as **Visual Studio Code** and **Anaconda Navigator**'s integrated environments, providing syntax highlighting, debugging, and terminal access.

- **Execution Environment:**
  The application runs in the **command-line interface (CLI) or terminal**, ensuring cross-platform
  compatibility without the need for graphical dependencies.

- **Libraries:**
  The project uses only **Python's standard libraries**, avoiding external dependencies for simplicity and ease of deployment.

RESULTS AND DISCUSSION

The Smart Recipe Recommender was tested with multiple input scenarios to evaluate accuracy and user experience.
When ingredients like 'rice, tomato, onion' were entered, the program suggested Indian dishes like 'Biryani' and 'Vegetable Pulao', as well as 'Mexican Rice' under Mexican cuisine.
The system displayed matched and missing ingredients clearly, allowing users to understand what could be cooked immediately and what was missing.
The algorithm handled both small and large ingredient lists efficiently, maintaining quick performance.
Even when minimal input was given, such as a single ingredient ('potato'), it returned valid recipes from multiple cuisines where the ingredient was used.
These results confirm that Python’s list and dictionary operations can be effectively applied for real-world recommendation systems.
The clean and logical structure makes the system easy to understand, modify, and enhance for future development.

1. Potential Improvements:
   - Adding a graphical user interface (GUI) to enhance user experience.
   - Providing various more input methods which enables ease to user.
   - AI implementation which leads to the providing of the procedure for the recipe.
   - Adding more features which leads to better understanding by the user for the recipe he wants to choose.

Applications:

The Smart Recipe Recommender has numerous practical and academic applications:

1. **Household Use:** Helps individuals quickly decide what to cook with available ingredients, reducing food wastage and saving time.

2. **Restaurants & Catering:** Useful for chefs to design menus based on inventory. It assists in identifying possible dishes using the ingredients left in stock.

3. **Educational Use:** Serves as an excellent Python learning project that applies data structures, loops, functions, and logic to real-life scenarios.

4. **Food Management & Health Systems:** Can be integrated with nutrition tracking systems to suggest balanced and healthy meals.

5. **Future AI Integration:** With future upgrades, it can evolve into a personalized meal planner using machine learning to remember user preferences and suggest diet-appropriate recipes.

Limitations:

While the Smart Recipe Recommender performs well, it has a few limitations:

1. **Text-Based Interface:** The project operates

entirely on a console interface, limiting its accessibility for non-technical users.

2. **Exact Keyword Matching:** Ingredient names must be entered exactly as they appear in the database. Variations like plural forms or misspellings may cause mismatches.

3. **No Personalization:** The current version does not store user preferences or learn from previous searches.

4. **Static Database:** The recipe dataset is predefined and cannot fetch live or trending recipes from the internet.

5. **No Nutritional or Calorie Data:** The project focuses solely on recipe suggestions without considering nutritional balance or calorie counts

Conclusion and Future Work:

The Smart Recipe Recommender successfully demonstrates a practical application of Python’s data structures, control flow, and functional programming principles.

It effectively provides a foundation for smart recipe recommendation systems and showcases how technology can simplify daily decision-making tasks.
Future developments can include artificial intelligence integration, web-based interfaces, nutritional tracking, and real-time recipe fetching for enhanced functionality.

REFERENCES

1. Python Software Foundation – Python Documentation (https://docs.python.org/3/)
2. GeeksforGeeks – Python Dictionary and Function Tutorials
3. W3Schools – Python Basics and Data Handling
4. Stack Overflow – Discussions on Ingredient Matching and Logic Implementation.

APPENDIX

Complete Python Code:

```
def get_recipes():
    """Extended and categorized recipe database (100+ recipes)"""
    return {
        # --- Indian Dishes ---
        "Indian": {
            "Aloo Gobi": ["potato", "cauliflower", "onion", "tomato", "turmeric", "garam masala"],
            "Paneer Butter Masala": ["paneer", "tomato", "butter", "cream", "garam masala", "onion"],
            "Dal Tadka": ["lentils", "onion", "tomato", "ghee", "garlic", "cumin"],
            "Chole": ["chickpeas", "onion", "tomato", "ginger", "garam masala", "cumin"],
            "Rajma": ["kidney beans", "onion", "tomato", "ginger", "garam masala", "garlic"],
            "Palak Paneer": ["spinach", "paneer", "garlic", "onion", "cream", "salt"],
            "Bhindi Masala": ["ladyfinger", "onion", "tomato", "garam masala", "turmeric"],
            "Vegetable Pulao": ["rice", "carrot", "peas", "onion", "cumin", "ghee"],
            "Masala Dosa": ["rice", "lentils", "potato", "onion", "turmeric", "mustard seeds"],
            "Biryani": ["rice", "chicken", "yogurt", "onion", "garam masala", "saffron"],
            "Butter Chicken": ["chicken", "tomato", "butter", "cream", "garam masala", "onion"],
            "Egg Curry": ["egg", "tomato", "onion", "garam masala", "ginger", "garlic"],
            "Fish Curry": ["fish", "coconut milk", "onion", "tomato", "chili", "turmeric"],
            "Poha": ["flattened rice", "onion", "mustard seeds", "turmeric", "lemon", "curry leaves"],
            "Upma": ["semolina", "onion", "mustard seeds", "carrot", "peas", "ghee"],
            "Pav Bhaji": ["potato", "cauliflower", "tomato", "onion", "butter", "spices"],
            "Kadhi Pakora": ["gram flour", "yogurt", "onion", "mustard seeds", "turmeric"],
            "Dhokla": ["gram flour", "yogurt", "turmeric", "mustard seeds", "curry leaves"],
        },

        # --- Italian Dishes ---
        "Italian": {
            "Pasta Alfredo": ["pasta", "cream", "butter", "cheese", "garlic", "salt"],
            "Pasta Arrabiata": ["pasta", "tomato", "olive oil", "garlic", "chili flakes", "basil"],
            "Lasagna": ["pasta sheets", "tomato sauce", "cheese", "beef", "onion", "oregano"],
            "Pizza Margherita": ["flour", "cheese", "tomato", "olive oil", "yeast", "basil"],
            "Mushroom Risotto": ["rice", "mushroom", "cheese", "butter", "onion", "white wine"],
            "Bruschetta": ["bread", "tomato", "garlic", "olive oil", "basil"],
            "Minestrone Soup": ["onion", "tomato", "beans", "pasta", "carrot", "celery"],
            "Spaghetti Carbonara": ["spaghetti", "egg", "bacon", "cheese", "pepper"],
            "Pesto Pasta": ["pasta", "basil", "olive oil", "garlic", "cheese", "pine nuts"],
            "Caprese Salad": ["tomato", "mozzarella", "basil", "olive oil", "salt"],
            "Gnocchi": ["potato", "flour", "egg", "cheese", "butter"],
            "Tiramisu": ["mascarpone", "coffee", "egg", "sugar", "cocoa powder"],
        },

        # --- Chinese Dishes ---
        "Chinese": {
            "Fried Rice": ["rice", "soy sauce", "carrot", "capsicum", "onion", "peas"],
            "Chow Mein": ["noodles", "soy sauce", "onion", "capsicum", "carrot", "garlic"],
            "Manchurian": ["cabbage", "carrot", "soy sauce", "cornflour", "garlic"],
            "Spring Rolls": ["flour", "cabbage", "carrot", "soy sauce", "onion"],
            "Hakka Noodles": ["noodles", "capsicum", "onion", "soy sauce", "vinegar"],
            "Schezwan Fried Rice": ["rice", "schezwan sauce", "garlic", "onion", "capsicum"],
            "Hot and Sour Soup": ["cabbage", "carrot", "vinegar", "soy sauce", "pepper"],
            "Sweet Corn Soup": ["corn", "milk", "onion", "pepper", "salt"],
            "Kung Pao Chicken": ["chicken", "soy sauce", "chili", "peanuts", "garlic"],
            "Mapo Tofu": ["tofu", "chili paste", "garlic", "soy sauce", "onion"],
```

```
    "Dim Sum": ["flour", "vegetables", "soy sauce", "ginger", "garlic"],
},

# --- Mexican Dishes ---
"Mexican": {
    "Tacos": ["tortilla", "chicken", "tomato", "lettuce", "cheese", "sauce"],
    "Burrito": ["tortilla", "rice", "beans", "chicken", "cheese", "salsa"],
    "Quesadilla": ["tortilla", "cheese", "onion", "capsicum", "sauce"],
    "Nachos": ["corn chips", "cheese", "jalapeno", "salsa", "beans"],
    "Guacamole": ["avocado", "onion", "tomato", "lemon", "salt"],
    "Enchiladas": ["tortilla", "chicken", "tomato sauce", "cheese", "beans"],
    "Fajitas": ["chicken", "onion", "capsicum", "spices", "tortilla"],
    "Churros": ["flour", "butter", "sugar", "cinnamon", "oil"],
    "Mexican Rice": ["rice", "tomato", "corn", "beans", "onion", "spices"],
},

# --- American Dishes ---
"American": {
    "Veg Burger": ["bun", "potato", "lettuce", "tomato", "cheese", "sauce"],
    "Cheeseburger": ["bun", "beef", "cheese", "lettuce", "tomato", "sauce"],
    "Grilled Sandwich": ["bread", "butter", "cheese", "tomato", "capsicum", "onion"],
    "Mac and Cheese": ["pasta", "cheese", "milk", "butter", "flour"],
    "Fried Chicken": ["chicken", "flour", "egg", "oil", "spices"],
    "Caesar Salad": ["lettuce", "chicken", "cheese", "croutons", "sauce"],
    "Omelette": ["egg", "onion", "tomato", "salt", "pepper"],
    "Pancakes": ["flour", "milk", "egg", "sugar", "butter", "baking powder"],
    "French Toast": ["bread", "egg", "milk", "sugar", "butter"],
    "BBQ Ribs": ["pork ribs", "bbq sauce", "garlic", "onion"],
    "Steak": ["beef", "salt", "pepper", "butter", "garlic"],
    "Cornbread": ["cornmeal", "flour", "milk", "egg", "sugar"],
},

# --- Japanese Dishes ---
"Japanese": {
    "Sushi": ["rice", "vinegar", "fish", "nori", "soy sauce"],
    "Ramen": ["noodles", "broth", "egg", "soy sauce", "chicken", "onion"],
    "Tempura": ["shrimp", "flour", "egg", "oil", "salt"],
    "Miso Soup": ["miso paste", "tofu", "seaweed", "onion"],
    "Teriyaki Chicken": ["chicken", "soy sauce", "garlic", "sugar", "ginger"],
    "Yakitori": ["chicken", "soy sauce", "skewers", "garlic", "sugar"],
    "Udon Noodles": ["udon noodles", "broth", "soy sauce", "spring onion"],
    "Okonomiyaki": ["flour", "cabbage", "egg", "soy sauce", "mayo"],
    "Onigiri": ["rice", "seaweed", "fish", "salt"],
    "Katsu Curry": ["chicken", "flour", "egg", "curry sauce", "rice"],
},

# --- Middle Eastern Dishes ---
"Middle Eastern": {
    "Falafel": ["chickpeas", "onion", "garlic", "parsley", "spices"],
    "Hummus": ["chickpeas", "tahini", "lemon", "garlic", "olive oil"],
    "Shawarma": ["chicken", "yogurt", "garlic", "spices", "pita"],
    "Tabbouleh": ["bulgur", "parsley", "lemon", "tomato", "olive oil"],
    "Kebabs": ["minced meat", "onion", "garlic", "spices", "bread"],
    "Baba Ganoush": ["eggplant", "tahini", "garlic", "lemon", "olive oil"],
    "Pita Bread": ["flour", "yeast", "salt", "water", "oil"],
    "Lamb Stew": ["lamb", "onion", "tomato", "garlic", "spices"],
    "Stuffed Grape Leaves": ["rice", "lemon", "onion", "grape leaves"],
},

# --- Desserts & Drinks ---
```

```
        "Desserts & Drinks": {
            "Gulab Jamun": ["milk powder", "flour", "sugar", "ghee", "cardamom"],
            "Kheer": ["rice", "milk", "sugar", "cardamom", "almonds"],
            "Halwa": ["semolina", "sugar", "ghee", "cardamom"],
            "Chocolate Cake": ["flour", "cocoa powder", "sugar", "egg", "butter", "milk"],
            "Lassi": ["yogurt", "sugar", "milk", "cardamom"],
            "Smoothie": ["milk", "banana", "sugar", "ice", "honey"],
            "Coffee": ["milk", "coffee powder", "sugar", "water"],
            "Tea": ["milk", "tea leaves", "sugar", "water", "ginger"],
            "Mango Shake": ["milk", "mango", "sugar", "ice"],
            "Ice Cream Sundae": ["ice cream", "chocolate syrup", "nuts", "whipped cream"],
            "Brownie": ["flour", "chocolate", "sugar", "egg", "butter"],
            "Cheesecake": ["cream cheese", "sugar", "egg", "butter", "biscuit base"],
        }
    }


def recommend_recipes(available_ingredients):
    """Recommend recipes from all cuisines"""
    cuisines = get_recipes()
    recommendations = {}

    for cuisine, recipes in cuisines.items():
        for recipe, ingredients in recipes.items():
            matched = [i for i in ingredients if i in available_ingredients]
            missing = [i for i in ingredients if i not in available_ingredients]
            if matched:
                recommendations[recipe] = {
                    "cuisine": cuisine,
                    "matched": matched,
                    "missing": missing
                }
    return recommendations


def main():
    print("🍽 Welcome to the Smart Recipe Recommender!")
    print("Enter the ingredients you have (comma separated):")

    user_input = input("> ").lower()
    available_ingredients = [x.strip() for x in user_input.split(",")]

    recommendations = recommend_recipes(available_ingredients)

    if not recommendations:
        print("\n❌No recipes found with those ingredients.")
    else:
        print("\n✅Recipes you can try:\n")
        for recipe, details in recommendations.items():
            print(f"🍴 {recipe} ({details['cuisine']})")
            print(f"   ✅Matched: {', '.join(details['matched'])}")
            print(f"   🛒 Missing: {', '.join(details['missing']) if details['missing'] else 'None'}")
            print("-" * 60)
```